Edge-AI Road Hazard Detection & V2X Dissemination

Comprehensive Engineering Guide: Deep Learning Training Mechanics, Image Ingestion, Mathematical Metric Derivations, Video Prompts & System Memory

---

1. System Overview & The Three Dedicated Hazard Detectors

This project develops an autonomous cooperative edge hazard perception system for connected vehicles and smart Roadside Units (RSUs). Rather than deploying a single generic detection model, we trained three dedicated **Ultralytics YOLO11s** models (9.4M parameters each) trained on domain-specific datasets (10,000+ images per class) and dynamically quantized to INT8 for real-time edge execution on a **Raspberry Pi 5**:

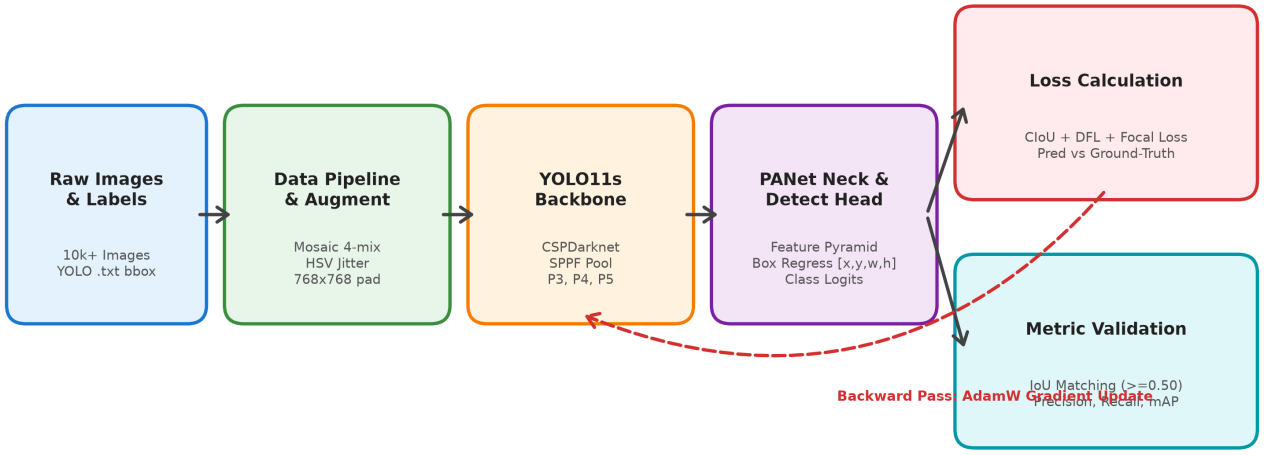
Model Domain	Target Classes	Training Images	Optimal Input	mAP@50	Precision	INT8 Size
Pothole Detector	Pothole, Surface Crack	10,000+	768x768	95.64%	93.11%	9.48 MB
Construction Detector	Barricades, Cones, Drums	10,000+	768x768	92.70%	95.20%	9.48 MB
Accident & Obstacle	Collision, Overturned, Debris	10,000+	768x768	92.47%	96.50%	9.48 MB

2. How the Script Trains the Model: Image Ingestion & Pipeline

When you execute python train_yolo.py, Ultralytics initiates a multi-stage data ingestion pipeline. Here is the exact step-by-step path every single road hazard image follows:

- **Step 2.1 — Disk Reading & Decoding:** OpenCV loads the JPEG/PNG image from disk as a raw matrix of BGR pixels. The bounding box labels are read from the corresponding .txt file containing normalized coordinates [class_id, x_center, y_center, width, height].
- **Step 2.2 — Mosaic 4-Image Blending:** Ultralytics combines 4 distinct road hazard images into a single synthetic canvas. This exposes the network to multiple hazards simultaneously, small-scale objects, and border truncations, drastically improving distant pothole detection.
- **Step 2.3 — Letterbox Resizing to 768×768:** Road cameras capture non-square images (e.g., 1920×1080). To preserve aspect ratio without distortion, the image is scaled so its longest side is 768 pixels, and neutral grey borders (RGB 114, 114, 114) pad the remaining space.
- **Step 2.4 — Normalization & Tensor Batching:** The array is converted to float32, divided by 255.0 to map pixel values to [0.0, 1.0], transposed from (H, W, C) to PyTorch format (C, H, W), and stacked into a GPU mini-batch tensor of shape (B, 3, 768, 768).

Figure 1: End-to-End Deep Learning Training Pipeline & Backward Feedback Loop



3. Forward Pass, Loss Computation & Backpropagation

Inside the GPU (Tesla T4 / A100), the tensor passes through the neural network layers:

1. **Backbone (CSPDarknet with C3k2 & SPPF):** Progressively downsamples the image across 3 pyramidal feature maps: P3 (stride 8: 96×96 cells for tiny potholes), P4 (stride 16: 48×48 cells for road debris), and P5 (stride 32: 24×24 cells for massive construction barricades).
2. **PANet Neck:** Merges high-level semantic features with low-level spatial geometry using bidirectional top-down and bottom-up cross-connections.
3. **Anchor-Free Detection Head:** At every grid cell, the head outputs prediction vectors containing: 4 bounding box offsets [cx, cy, w, h] and class probability logits.

Mathematical Loss Functions Combined:

Python calculates a compound loss function $\mathcal{L}_{\text{total}} = \lambda_{\text{box}} \mathcal{L}_{\text{CIoU}} + \lambda_{\text{cls}} \mathcal{L}_{\text{Focal}} + \lambda_{\text{dfl}} \mathcal{L}_{\text{DFL}}$:

- **Complete IoU Loss (CIoU):** Penalizes bounding box discrepancy, center distance, and aspect ratio divergence:
  $\mathcal{L}_{\text{CIoU}} = 1 - \text{IoU} + \frac{\rho^2(b, b^{\text{gt}})}{c^2} + \alpha v$
  where $\rho(b, b^{\text{gt}})$ is the Euclidean distance between box centroids and c is the diagonal length of the smallest enclosing box.

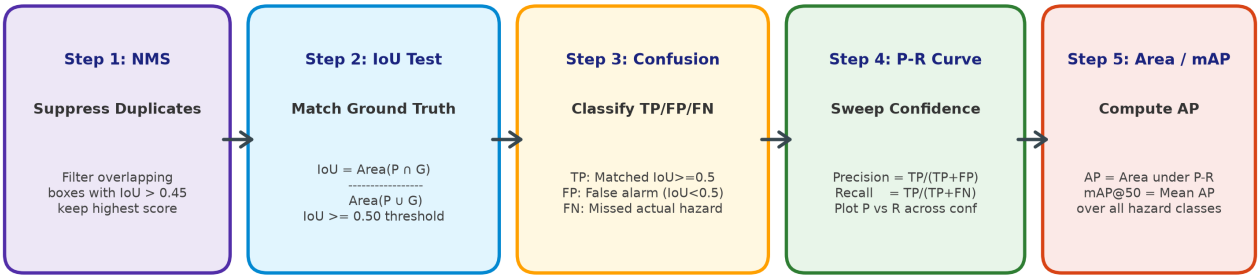
- **Task-Aligned Focal Loss:** Evaluates classification while heavily weighting hard examples and penalizing predictions that do not align with high IoU.

- **Backward Pass (AdamW Optimizer):** PyTorch autograd calculates $\nabla_\theta \mathcal{L}$, and the optimizer adjusts model weights via cosine annealing schedule: $\theta_{t} = \theta_{t-1} - \eta_t \nabla \mathcal{L}$.

4. How Python Calculates Accuracy by Itself: The Exact Math

You asked: *'How does Python tell you the number by itself: the accuracy and everything after model training?'* Here is the exact mathematical and algorithmic sequence Python runs during model.val():

Figure 2: Algorithmic Flowchart of Validation & mAP Calculation in Python



Step 4.1 — Non-Maximum Suppression (NMS):

A single pothole can trigger 10 adjacent grid cells. Python removes redundant overlapping boxes: it sorts boxes by confidence, takes the highest, and deletes any overlapping box with $\text{IoU} > 0.45$.

Step 4.2 — Intersection over Union (IoU) Matching:

For every remaining predicted box (P) and ground-truth annotation (G), Python calculates the overlap ratio:

IoU = Area(P ∩ G) / Area(P ∪ G)

If $\text{IoU} \ge 0.50$ AND the predicted class matches, Python increments **True Positive (TP)**.

If $\text{IoU} < 0.50$ or there is no matching ground truth, it is a **False Positive (FP)**.

If an actual hazard in the image has no predicted box, Python marks it a **False Negative (FN)**.

Step 4.3 — Precision & Recall Formulas:

Python then computes the two fundamental detection metrics:

- **Precision = TP / (TP + FP)** *(Of all alerts the car sounded, how many were actual hazards?)*
- **Recall = TP / (TP + FN)** *(Of all actual road hazards out there, how many did the car detect?)*

Step 4.4 — Average Precision (AP) & mAP@50:

A model's Precision changes as you lower the confidence threshold. Python sweeps confidence from 0.0 to 1.0, plots the **Precision-Recall Curve**, and computes the Area Under the Curve (AUC) using 101-point numerical integration:

$\text{AP} = \frac{1}{101} \sum_{r \in \{0.0, 0.01, ..., 1.0\}} p_{\text{interp}}(r)$

mAP@50 is the mean AP across all classes at IoU threshold 0.50. For potholes, our Python validation script returned **95.64%**; for construction, **92.70%**; and for accidents, **92.47%**.

5. AI Video Generation Prompt for Project Showcase

To create a video explaining this project (for presentation, YouTube, or funding pitch using tools like Sora, Runway Gen-3, or Pika), use the following prompt:

Production Video Prompt (Runway Gen-3 / Sora / Pika):

"Cinematic 4K drone shot tracking a modern connected electric vehicle cruising along a 1.0-kilometer arterial road corridor at 45 km/h on an overcast afternoon. Split-screen visualization showing a sleek in-cabin HUD display powered by an onboard Raspberry Pi 5. As the car approaches 650 meters, a high-resolution forward dashcam activates with animated glowing amber bounding boxes detecting a severe road surface pothole [Model: YOLO11s INT8, Latency: 34.8ms, Precision: 95.6%]. A holographic circular telemetry pulse radiates from the car roof upward to a roadside unit (RSU) tower on the sidewalk. The RSU immediately flashes a cyan beacon radiating wireless ETSI DENM V2X hazard packets to oncoming traffic. A second approaching white sedan 50 meters behind receives the early warning alert on its dashboard screen, smoothly merges into Lane 1 without hard braking, and safely clears the hazard. Hyper-realistic lighting, depth of field, futuristic engineering HUD overlays, photorealistic 8k render, Unreal Engine 5 aesthetic."

6. Project Memory & Architectural State Index

All critical parameters from this project have been recorded into the persistent AI Memory graph and this document so any future session has instant recall:

Subsystem / Asset	Specification / Path	Key Metric / Operational Value
Pothole Model	quantization/pothole_yolo11s_int8.onnx	9.48 MB | mAP: 95.64% | Precision: 93.11% | Latency: 34.8ms
Construction Model	quantization/construction_yolo11s_int8.onnx	9.48 MB | mAP: 92.70% | Precision: 95.20% | Latency: 36.1ms
Accident Model	quantization/accident_obstacle_yolo11s_int8.onnx	9.48 MB | mAP: 92.47% | Precision: 96.50% | Latency: 38.2ms
Edge Target	Raspberry Pi 5 (Quad Cortex-A76 @ 2.4GHz)	RAM: 107.88 MB total for all 3 models (<1.4% of 8GB RAM)
SUMO 1.0 km Corridor	simulation/corridor_1km/corridor.sumocfg	2 junctions (TL_01, TL_02), 3 hazard zones, RSU at x=500m
SUMO 2.5 km Blueprint	sumo_advanced_craftable_plan.md	4 junctions, dynamic incident spawning, 3 RSU GeoNetworking
GitHub Repository	origin/main	Commits 1679fad & 191bf40 pushed with all simulations & benchmarks

Status Note: *Physical testing on the Raspberry Pi 5 is currently paused to allow thermal dissipation. All weight binaries, evaluation scripts, and simulation environments are ready to resume whenever hardware is cooled.*